API Reference

Signaling Server API

REST API Endpoints

Health Check

GET `/health`

Check server health status.

Response:

```
{
  "status": "healthy",
  "timestamp": "2024-01-01T12:00:00.000Z",
  "uptime": 3600
}
```

Server Statistics

GET `/api/stats`

Get server statistics.

Response:

```
{
  "activeSessions": 5,
  "connectedClients": 3,
  "totalVMs": 5,
  "uptime": 86400
}
```

Session Lookup

GET `/api/session/:sessionId`

Lookup session by ID.

Parameters:
- `sessionId` (string, required) - Session UUID

Response (Found):

```
{
  "found": true,
  "session": {
    "sessionId": "550e8400-e29b-41d4-a716-446655440000",
    "vmId": "gaming-vm-001",
    "socketId": "abc123",
    "registeredAt": "2024-01-01T12:00:00.000Z",
    "lastHeartbeat": "2024-01-01T12:30:00.000Z",
    "status": "active",
    "metadata": {
      "hostname": "gaming-server-01",
      "ip": "10.0.1.5",
      "capabilities": {
        "maxResolution": "1920x1080",
        "maxBitrate": 8000,
        "codecs": ["H264", "VP8"]
      }
    }
  }
}
```


Response (Not Found):


```
{
  "found": false,
  "message": "Session not found or expired"
}
```

List Active Sessions

GET `/api/sessions`

Get all active sessions.

Response:


```
{
  "count": 2,
  "sessions": [
    {
      "sessionId": "550e8400-e29b-41d4-a716-446655440000",
      "vmId": "gaming-vm-001",
      "status": "active",
      "registeredAt": "2024-01-01T12:00:00.000Z"
    },
    {
      "sessionId": "660e8400-e29b-41d4-a716-446655440001",
      "vmId": "gaming-vm-002",
      "status": "connected",
      "registeredAt": "2024-01-01T12:05:00.000Z"
    }
  ]
}
```

Socket.IO Events

VM Namespace (`/vm`)

Events for VM host connections.

VM → Server: `register`

Register VM and get session ID.

Payload:

```
{
  vmId: string;
  hostname?: string;
  ip?: string;
  capabilities?: {
    maxResolution: string;
    maxBitrate: number;
    codecs: string[];
  };
}
```

Response:

```
{
  success: boolean;
  sessionId?: string;
  message: string;
}
```

Example:

```
// VM sends
socket.emit('register', {
  vmId: 'gaming-vm-001',
  hostname: 'gaming-server-01',
  ip: '10.0.1.5',
  capabilities: {
    maxResolution: '1920x1080',
    maxBitrate: 8000,
    codecs: ['H264', 'VP8']
  }
}, (response) => {
  console.log('Session ID:', response.sessionId);
});
```

VM → Server: `heartbeat`

Send periodic heartbeat.

Payload:

```
{
  sessionId: string;
}
```

Response:

```
{
  success: boolean;
}
```

Server → VM: `offer`

Receive WebRTC offer from client.

Payload:

```
{
  sessionId: string;
  offer: RTCSessionDescriptionInit;
}
```

VM → Server: `answer`

Send WebRTC answer to client.

Payload:

```
{
  sessionId: string;
  answer: RTCSessionDescriptionInit;
}
```

Bidirectional: `ice-candidate`

Exchange ICE candidates.

Payload:

```
{
  sessionId: string;
  candidate: RTCIceCandidateInit;
}
```

Client Namespace (`/client`)

Events for web client connections.

Client → Server: `join-session`

Join a streaming session.

Payload:

```
{
  sessionId: string;
}
```

Response:

```
{
  success: boolean;
  message: string;
  session?: SessionInfo;
}
```

Example:

```
// Client sends
socket.emit('join-session', {
  sessionId: '550e8400-e29b-41d4-a716-446655440000'
}, (response) => {
  if (response.success) {
    console.log('Joined session:', response.session);
  } else {
    console.error('Failed:', response.message);
  }
});
```

Client → Server: `offer`

Send WebRTC offer.

Payload:

```
{
  sessionId: string;
  offer: RTCSessionDescriptionInit;
}
```

Server → Client: `answer`

Receive WebRTC answer.

Payload:

```
{
  answer: RTCSessionDescriptionInit;
}
```

Bidirectional: `ice-candidate`

Exchange ICE candidates.

Payload:

```
{
  sessionId: string;
  candidate: RTCIceCandidateInit;
}
```

WebRTC Data Channel Protocol

Message Format

All data channel messages use JSON format:

```
{
  type: string;      // Message type
  data?: any;        // Message data
  timestamp: number; // Unix timestamp
}
```

Message Types

Input Message (Client → VM)

Type: `input`

Send user input to VM.

Payload:

```
{
  type: "input",
  data: {
    type: "keyboard" | "mouse_move" | "mouse_button" | "mouse_wheel" | "gamepad";
    // ... type-specific fields
  },
  timestamp: number;
}
```

Examples:

```
// Keyboard
{
  type: "input",
  data: {
    type: "keyboard",
    key: "w",
    code: "KeyW",
    action: "press"
  },
  timestamp: 1640000000000
}

// Mouse movement
{
  type: "input",
  data: {
    type: "mouse_move",
    dx: 10,
    dy: -5
  },
  timestamp: 1640000000001
}

// Mouse button
{
  type: "input",
  data: {
    type: "mouse_button",
    button: "left",
    action: "press"
  },
  timestamp: 1640000000002
}

// Mouse wheel
{
  type: "input",
  data: {
    type: "mouse_wheel",
    dx: 0,
    dy: 100
  },
  timestamp: 1640000000003
}

// Gamepad
{
  type: "input",
  data: {
    type: "gamepad",
    action_type: "button",
    button: 0,
    pressed: true,
    value: 1.0
  },
  timestamp: 1640000000004
}
```

Quality Change (Client → VM)

Type: `quality_change`

Request quality preset change.

Payload:

```
{
  type: "quality_change",
  quality: "high" | "medium" | "low",
  timestamp: number;
}
```

Example:

```
{
  type: "quality_change",
  quality: "medium",
  timestamp: 1640000000005
}
```

Stats Request (Client → VM)

Type: `stats_request`

Request connection statistics.

Payload:

```
{
  type: "stats_request",
  timestamp: number;
}
```

TypeScript Type Definitions

Session Types

```typescript
interface VMSession {
  sessionId: string;
  vmId: string;
  socketId: string;
  registeredAt: Date;
  lastHeartbeat: Date;
  status: 'active' | 'connected' | 'disconnected';
  metadata?: {
    hostname?: string;
    ip?: string;
    capabilities?: {
      maxResolution: string;
      maxBitrate: number;
      codecs: string[];
    };
  };
}

interface ClientConnection {
  socketId: string;
  sessionId: string;
  connectedAt: Date;
  status: 'connecting' | 'connected' | 'disconnected';
}
```

Input Types

```typescript
interface KeyboardInput {
  type: 'keyboard';
  key: string;
  code: string;
  action: 'press' | 'release';
}

interface MouseMoveInput {
  type: 'mouse_move';
  dx: number;
  dy: number;
}

interface MouseButtonInput {
  type: 'mouse_button';
  button: 'left' | 'middle' | 'right';
  action: 'press' | 'release';
}

interface MouseWheelInput {
  type: 'mouse_wheel';
  dx: number;
  dy: number;
}

interface GamepadInput {
  type: 'gamepad';
  action_type: 'button' | 'axis';
  button?: number;
  axis?: number;
  pressed?: boolean;
  value?: number;
}

type InputEvent =
  | KeyboardInput
  | MouseMoveInput
  | MouseButtonInput
  | MouseWheelInput
  | GamepadInput;
```

Connection Stats

```typescript
interface ConnectionStats {
  latency: number;      // Round-trip time in ms
  bitrate: number;      // Current bitrate in kbps
  packetLoss: number;   // Packet loss count
  frameRate: number;    // Current FPS
  resolution: string;   // Current resolution (e.g., "1920x1080")
}
```

Quality Presets

```typescript
interface QualityPreset {
  name: 'high' | 'medium' | 'low';
  width: number;
  height: number;
  fps: number;
  bitrate: number;
}

const QUALITY_PRESETS: Record<string, QualityPreset> = {
  high: {
    name: 'high',
    width: 1920,
    height: 1080,
    fps: 60,
    bitrate: 8000
  },
  medium: {
    name: 'medium',
    width: 1280,
    height: 720,
    fps: 60,
    bitrate: 4000
  },
  low: {
    name: 'low',
    width: 960,
    height: 540,
    fps: 60,
    bitrate: 2000
  }
};
```

Error Codes

REST API Errors

Code	Message	Description
404	Session not found	Session ID doesn’t exist or has expired
500	Internal server error	Server encountered an error

Socket.IO Errors

Event	Error	Description
connect_error	Connection failed	Cannot connect to signaling server
join-session	Session not found	Invalid or expired session ID
join-session	Session not active	VM is not ready for connections

WebRTC Errors

Error	Description	Solution
OperationError	Failed to create offer	Check browser compatibility
InvalidStateError	Connection in wrong state	Restart connection
NotFoundError	No media tracks	VM not streaming

Rate Limits

Current implementation has no rate limits. Recommended for production:

Endpoint	Limit	Window
`/api/*`	100 requests	1 minute
Socket connect	10 connections	1 minute per IP
Session lookup	50 requests	1 minute

Versioning

API Version: **1.0.0**

Current implementation does not include API versioning. For future versions, use:

```
GET /v1/api/stats
GET /v2/api/stats
```

WebSocket Connection URLs

Development

```
http://localhost:3000/vm
http://localhost:3000/client
```

Production

```
https://signaling.yourdomain.com/vm
https://signaling.yourdomain.com/client
```

CORS Configuration

Allowed origins are configured via environment variable:

```
ALLOWED_ORIGINS=https://gaming.yourdomain.com,https://www.yourdomain.com
```

Methods allowed:
- GET
- POST
- OPTIONS

Credentials: Enabled

Security Headers

Response headers include:

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

Note: CSP is disabled for WebRTC compatibility.